Algorithm Efficiency

Peerapon S.

CPE 231 Algorithms

Topics

Analysis of running times

Growth order of running time

Asymptotic efficiency classes

Running Time and Input Size

Logical to measure algorithm efficiency using ***running time.***

Almost all algorithms run longer on larger ***input size***

Time efficiency



Memory (space) efficiency

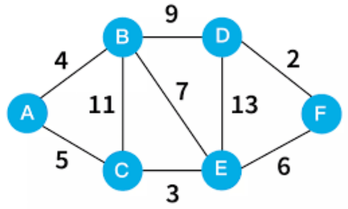


Characterizing Input Size

Sorting a list of numbers $\{a_1, a_2, \cdots, a_n\}$?

Multiplying two integers a_1 and a_2 ?

Finding a shortest path in a graph ?





Determining if a given positive integer is prime ?

Input Size ≠ Input Value !

Running Time Approximation using Count of Basic Operations $C(n)$

Why not just measuring code execution time?

Count of Basic operation $C(n)$

> Input-related operation contributing most to the total running time in a (nested) loop.

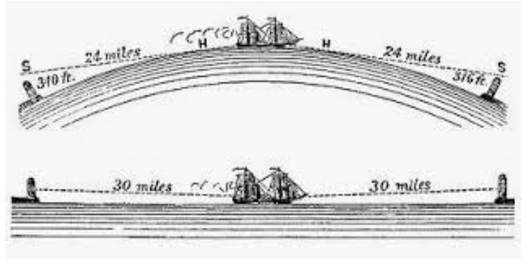
> Usually the most time-consuming operation in the innermost loop.

> Used as a proxy measure of running time.

==, <, >, ≤, ≥

÷, ×, +, −

Can also count other quantities, e.g., key swap or movement, when the memory/disk access time is of interest.



But shouldn't × take longer than + ?



Ex: MaxElement()

What is the basic operation in MaxElement() ?

Determine $C(n)$ of MaxElement().

Algorithm MaxElement

1: Input: A sequence of numbers $a_1, a_2, \cdots, a_n$
2: Output: The largest element in the input sequence
3:
4: $maxval \leftarrow a_1$
5: **for** $i = 2$ to n **do**
6: **if** $maxval < a_i$ **then**
7: $maxval \leftarrow a_i$
8: **return** $maxval$

Steps for Analyzing $C(n)$ of Non-Recursive Algorithms

Step	Ex: MaxElement()
Decide on parameters indicating the input size.	
Identify algorithm's basic operations.	
Check if the execution of basic operations depends on additional properties other than the input size.	
Set up the sum expressing # executions of basic operations.	
Evaluate the sum to closed-form (and establish the growth order).	

Useful summations

1. $\sum_{i=l}^{u} 1 = \underbrace{1+1+\cdots+1}_{u-l+1 \text{ times}} = u-l+1$ (l, u are integer limits, $l \le u$); $\sum_{i=1}^{n} 1 = n$

2. $\sum_{i=1}^{n} i = 1+2+\cdots+n = \frac{n(n+1)}{2} \approx \frac{1}{2}n^2$

3. $\sum_{i=1}^{n} i^2 = 1^2+2^2+\cdots+n^2 = \frac{n(n+1)(2n+1)}{6} \approx \frac{1}{3}n^3$

4. $\sum_{i=1}^{n} i^k = 1^k+2^k+\cdots+n^k \approx \frac{1}{k+1}n^{k+1}$

5. $\sum_{i=0}^{n} a^i = 1+a+\cdots+a^n = \frac{a^{n+1}-1}{a-1}$ ($a \ne 1$); $\sum_{i=0}^{n} 2^i = 2^{n+1}-1$

6. $\sum_{i=1}^{n} i2^i = 1\cdot 2+2\cdot 2^2+\cdots+n2^n = (n-1)2^{n+1}+2$

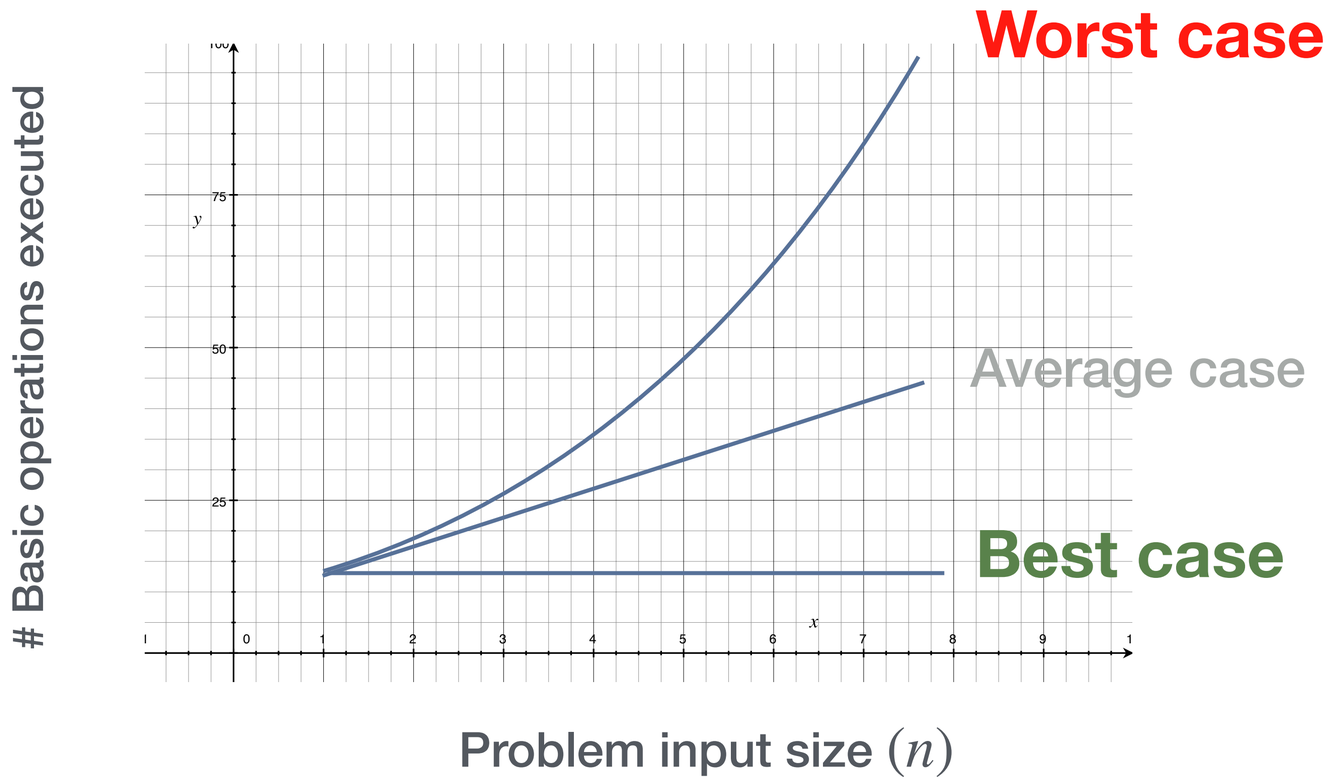
7. $\sum_{i=1}^{n} \frac{1}{i} = 1+\frac{1}{2}+\cdots+\frac{1}{n} \approx \ln n+\gamma$, where $\gamma \approx 0.5772\ldots$ (Euler's constant)

8. $\sum_{i=1}^{n} \lg i \approx n \lg n$

Best / Average / Worst-case Running Time

Running time or $C(n)$ may depend on the input characteristic in addition to the input size.

Desirable to know how an algorithm works over all input instances.



Example

From the algorithm below,

- What task does it solve ?
- What is the basic operation in the algorithm ?
- Are there best-case and worst-case $C(n)$? When do they happen?
  If so, determine both functions.

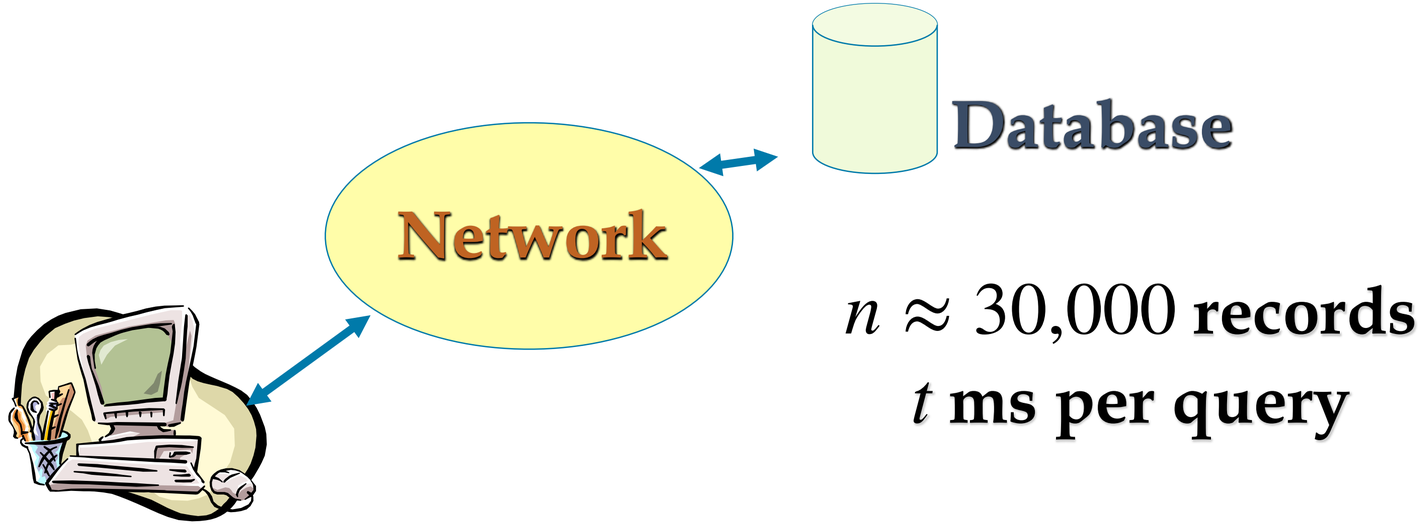
Algorithm UNKNOWN

1: **Input**: A sequence of numbers $a_1, a_2, \cdots, a_n$
2: **Output**: ----------
3:
4: **for** $i = 1$ to $n-1$ **do**
5: **for** $j = i+1$ to n **do**
6: **if** $a_i == a_j$ **then**
7: **return** *false*
8: **return** *true*

Step	UNKNOWN()
Decide on parameters indicating an input's size.	
Identify algorithm's basic operations	
Check if the execution of basic operations depends on additional properties other than the input size	
Set up the sum expressing # executions of basic operations	
Evaluate the sum to closed-form	

In-Class Assignment L2-1: Running Time Analysis (Group of 2 to 3)

(25 minutes) Submit to LEB2



$n \approx 30{,}000$ **records**

t **ms per query**

How long to wait on average to search for a given record ?

No. of records (n)	Waiting time (10 ms per lookup)		Waiting time (1 ms per lookup)	
	Linear search (n)	Binary search ($\log_2 n$ / lg n)	Linear search (n)	Binary search ($\log_2 n$ / lg n)
n = 1000	10 s	0.1 s	1 s	0.01 s
n = 10000	100 s	0.1329 s	10 s	0.01329 s
n = 30000	300	0.1487 s	30 s	0.01487 s

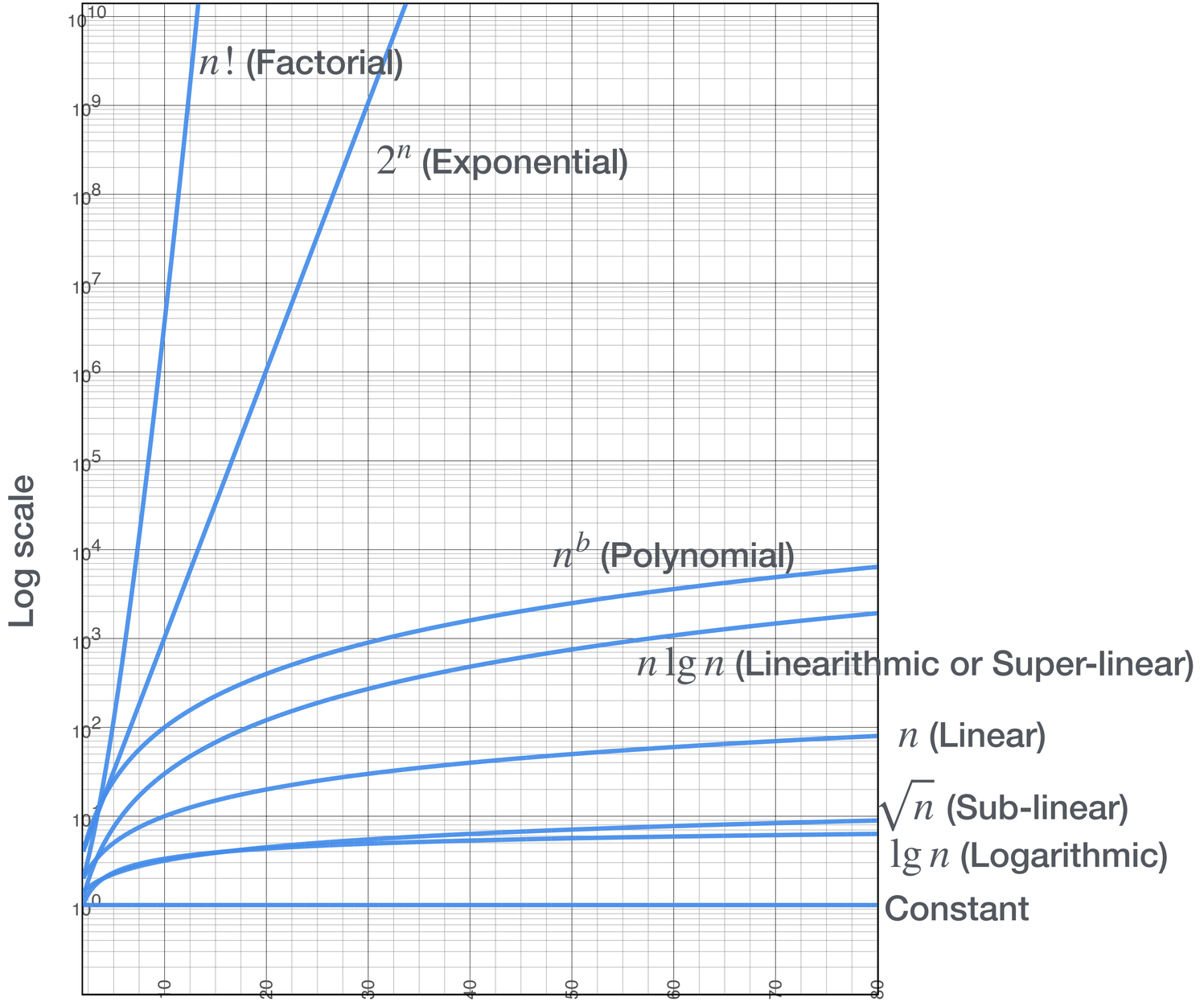
Growth Order

How does $C(n)$ behave when n increases?

Which term in $C(n)$ dominates as n gets large?

n	$C(n) = n^2/4 + n/2$
20	
200	
2000	
20000	

Common Growth Order



Why Algorithm Growth Order Matters?

Assume 1 ns per step

n $f(n)$	$\lg n$	n	$n \lg n$	n^2	2^n	$n!$
10	0.003 μs	0.01 μs	0.033 μs	0.1 μs	1 μs	3.63 ms
20	0.004 μs	0.02 μs	0.086 μs	0.4 μs	1 ms	77.1 years
30	0.005 μs	0.03 μs	0.147 μs	0.9 μs	1 sec	8.4×10^{15} yrs
40	0.005 μs	0.04 μs	0.213 μs	1.6 μs	18.3 min	
50	0.006 μs	0.05 μs	0.282 μs	2.5 μs	13 days	
100	0.007 μs	0.1 μs	0.644 μs	10 μs	4×10^{13} yrs	
1,000	0.010 μs	1.00 μs	9.966 μs	1 ms		
10,000	0.013 μs	10 μs	130 μs	100 ms		
100,000	0.017 μs	0.10 ms	1.67 ms	10 sec		
1,000,000	0.020 μs	1 ms	19.93 ms	16.7 min		
10,000,000	0.023 μs	0.01 sec	0.23 sec	1.16 days		
100,000,000	0.027 μs	0.10 sec	2.66 sec	115.7 days		
1,000,000,000	0.030 μs	1 sec	29.90 sec	31.7 years		

Asymptotic Efficiency Class

Formalize the concept of growth order to allow for classifying algorithms

Provide ***asymptotic bounds*** (exact, upper, lower) on the running time of algorithms (worst, best, average cases)

- Big-Theta (Θ) notation
- Big-Oh (O) notation
- Big-Omega (Ω) notation

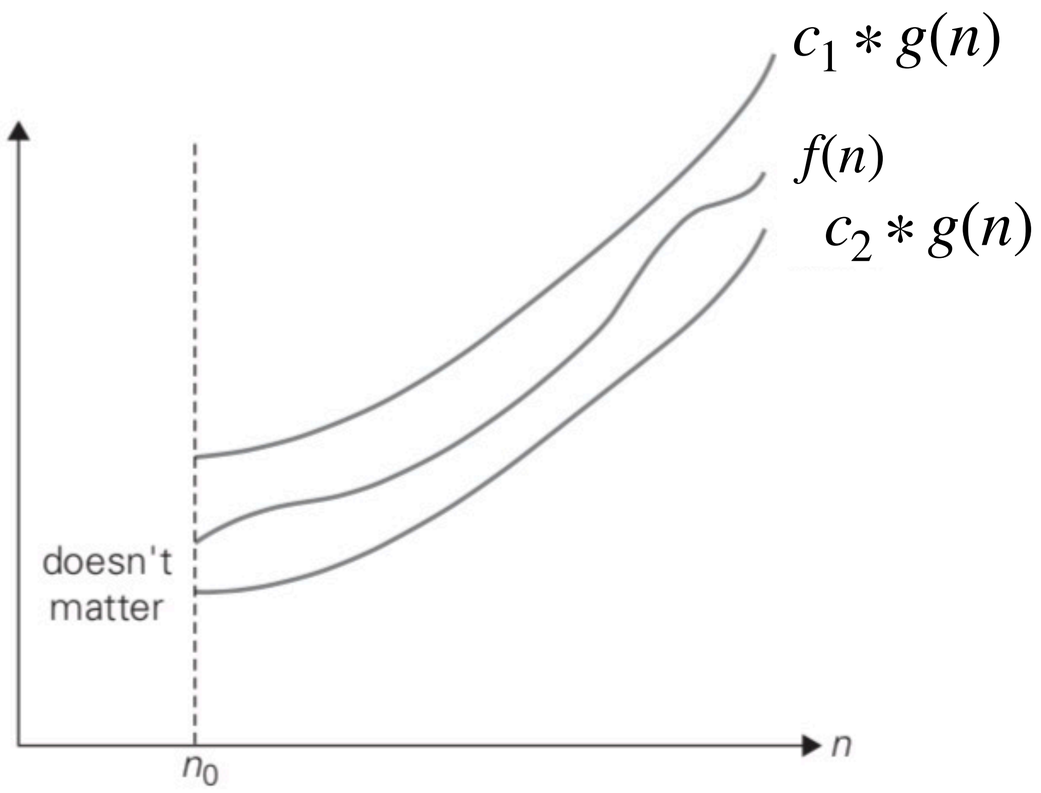
The choice of notation depends on how you derive the running time.

Different algorithms solving the same problem can be ranked and compared.

Big-Θ Notation

$\Theta(g(n))$ is the set of all functions with the same growth order as $g(n)$ within a constant multiple as $n \to \infty$.

Asymptotically Exact/Tight bound



$$f(n) \in \Theta(g(n))$$

$$c_2 g(n) \leq f(n) \leq c_1 g(n), \ \forall n \geq n_0$$

- $f(n)$ is big-Theta of $g(n)$.
- $f(n)$ grows precisely as $g(n)$.
- $f(n)$ has the growth order bounded above and below by $g(n)$.

Example

Let $C(n) = 8n \lg n + 3n^2$. Is $C(n) \in \Theta(n^2)$?

For a formal proof, we need to show that there exists c_1, c_2, n_0 such that

$$c_2 n^2 \leq C(n) \leq c_1 n^2, \ \forall n \geq n_0$$

Asymptotic Notation in Equations

Can use $=$ instead of $\in$ in a statement.

$$4n^2 + 100n = \Theta(n^2)$$

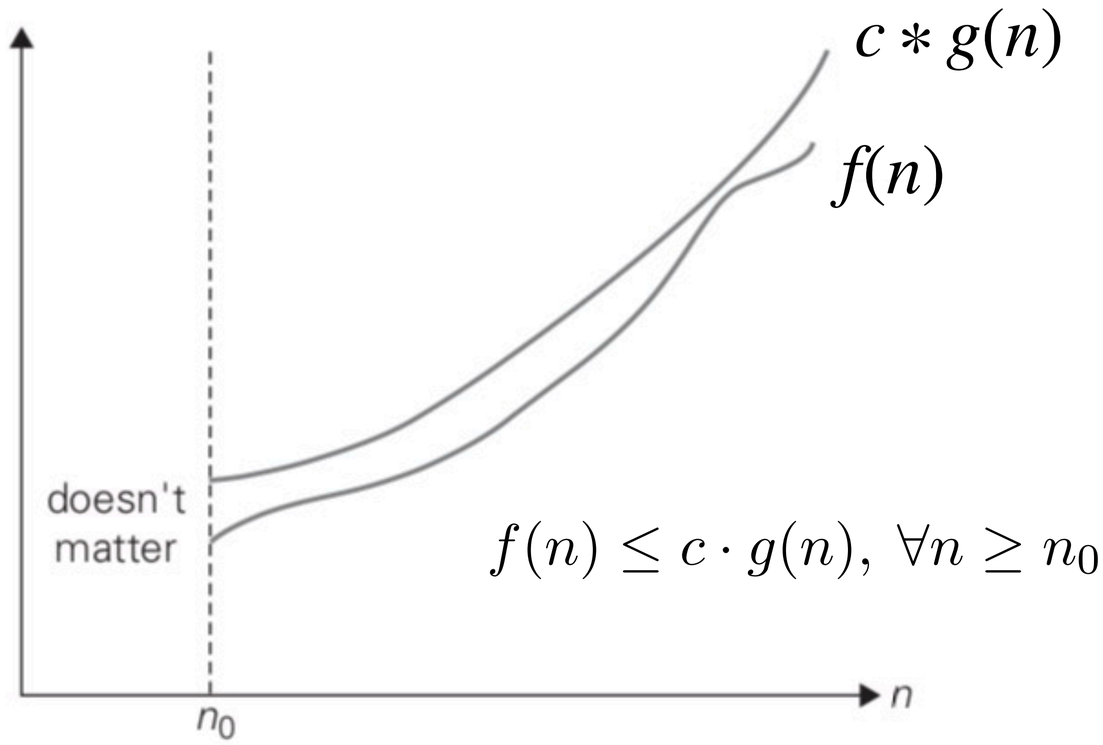
$$2n^2 + 3n + 1 = 2n^2 + \Theta(n)$$

$$2n^2 + \Theta(n) = \Theta(n^2)$$

Big-O Notation

$O(g(n))$ is the set of all functions with a smaller or same growth order as $g(n)$ within a constant multiple as $n \to \infty$.

Asymptotically Upperbound

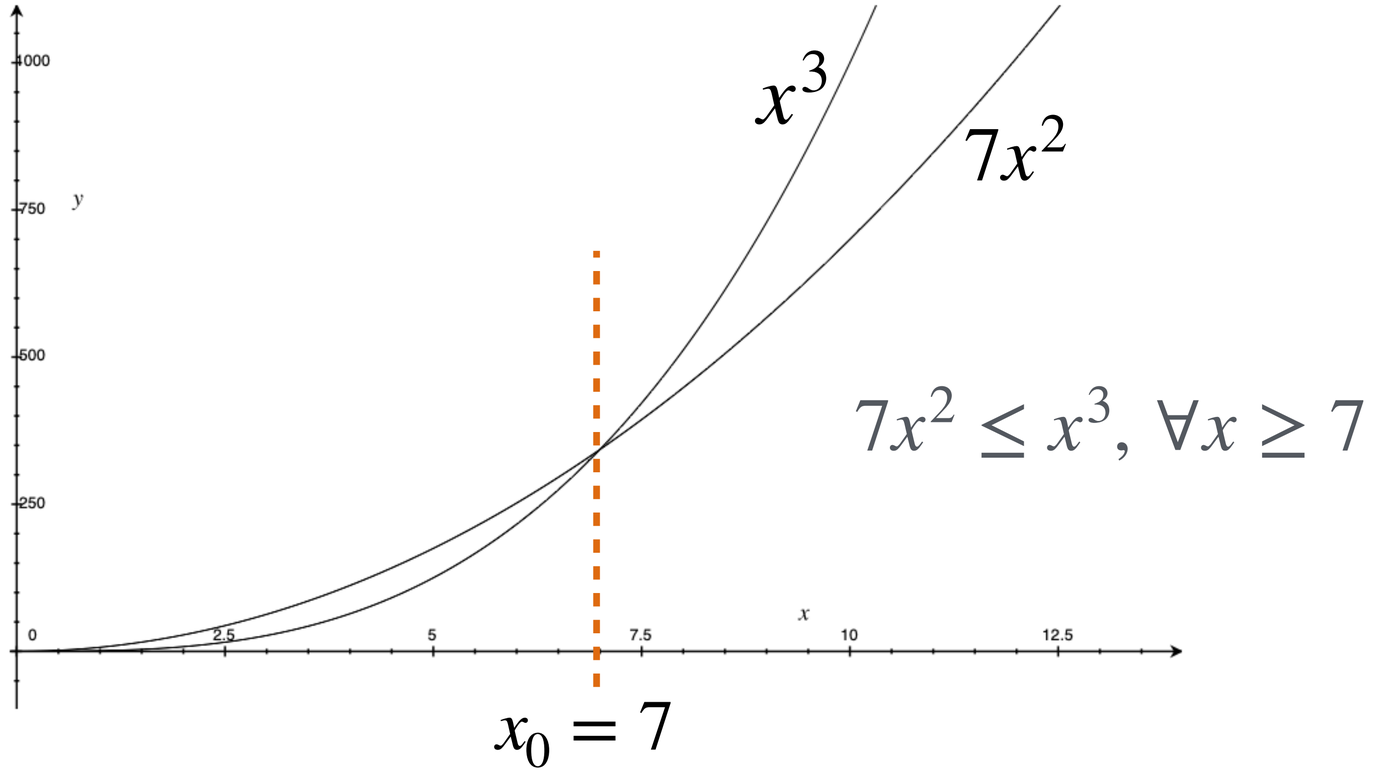


$$f(n) \in O(g(n))$$

- $f(n)$ is big-O of $g(n)$.
- $f(n)$ grows no faster than $g(n)$.
- $f(n)$ has the growth order bounded above by $g(n)$.
- $O(g(n))$ is a **big-Oh estimate** of $f(n)$.

Ex:

- Is $f(x) = 7x^2 \in O(x^3)$?
- Is $f(x) = 7x^2 \in O(x^2)$?



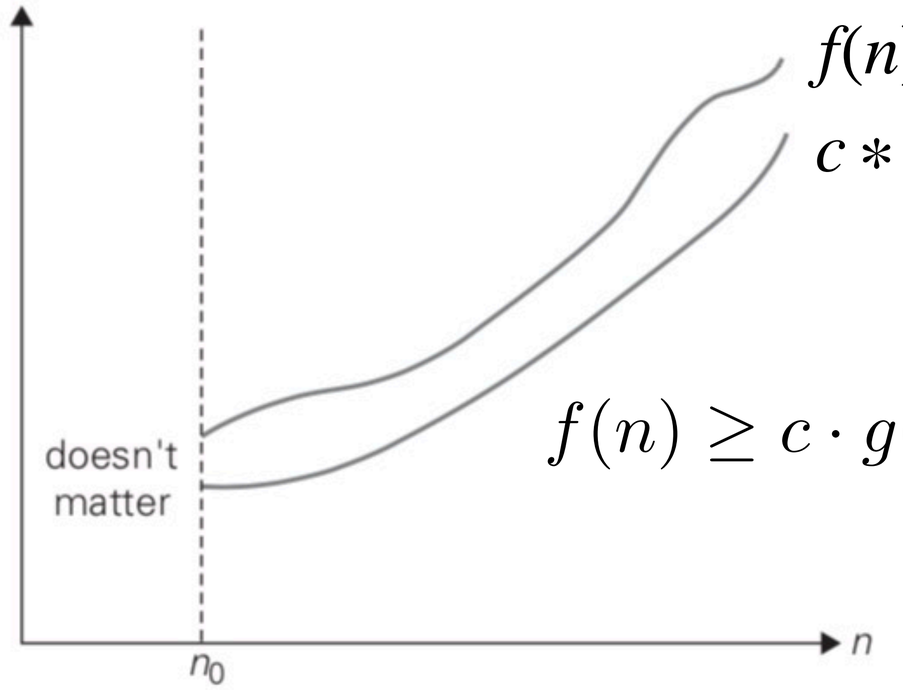
Which one above should be used ?

- Asymptotic notation should provide the ***simplest*** and ***most precise bound*** possible.
- Should not use $O(n^3)$ when $O(n^2)$ can be used.
- Should not use $O(n^2)$ when $\Theta(n^2)$ can be used.

Big-Ω Notation

$\Omega(g(n))$ is the set of all functions with a larger or same growth order as $g(n)$ within a constant multiple as $n \to \infty$.

Big-Ω Notation: Asymptotically Lowerbound



$$f(n) \geq c \cdot g(n),\ \forall n \geq n_0$$

$$f(n) \in \Omega(g(n))$$

- $f(n)$ is big-Omega of $g(n)$.
- $f(n)$ grows at least as fast as $g(n)$.
- $f(n)$ has the growth order bounded below by $g(n)$.
- $\Omega(g(n))$ is a **big-Omega estimate** of $f(n)$.

Algorithm (Time) Complexity/Efficiency

If the best-case, average-case, or worst-case algorithm running time $f(n)$ is $\Theta(g(n))$ or $O(g(n))$, the algorithm is respectively said to have the ***complexity / asymptotic efficiency*** of $\Theta(g(n))$ or $O(g(n))$ in best-case, average-case, worst-case.

g(n)	Θ(g(n))	O(g(n))	Efficiency Class
1	Θ(1)	O(1)	Constant
lg n	Θ(lg n)	O(lg n)	Logarithmic
n	Θ(n)	O(n)	Linear
n lg n	Θ(n lg n)	O(n lg n)	Linearithmic / Superlinear
√n	Θ(√n)	O(√n)	Sublinear polynomial
n^b	Θ(n^b)	O(n^b)	Polynomial
b^n	Θ(b^n)	O(b^n)	Exponential
n!	Θ(n!)	O(n!)	Factorial

Do the following two statements have the same interpretation?

I: Algorithm `foo()` is $O(n^2)$, i.e., an $O(n^2)$ algorithm.

II: Algorithm `foo()` is $\Theta(n^2)$, i.e., an $\Theta(n^2)$ algorithm.

Self-Test: Algorithm Efficiencies



https://forms.office.com/r/XgStNR5ekX?origin=lprLink

Useful Property of Asymptotic Notations

Part A: $C_1(n) \in O(n)$



Part B: $C_2(n) \in O(n \lg n)$

What is the asymptotic bound of $C_1(n) + C_2(n)$?

Part A: $C_1(n) \in O(n)$



Part B: $C_2(m) \in O(m)$

What is the asymptotic bound of $C_1(n) + C_2(m)$?

Summary

Algorithm efficiency is measured by running time estimated from the number of basic operations executed.

Running time on large inputs distinguishes efficient algorithms from inefficient ones.

Growth order of running time indicates the ***asymptotic efficiency*** of the algorithm

Asymptotic notations - Big-O and friends

- Ignore tedious details and focus on how the running time grows with n.
- Classifying algorithms into efficiency classes.
- Beware of the "constant factors"